

Java程序设计



授课人：何毅
办公室：1909



CITY INSTITUTE
城市学院

第12讲 内容向导

12.1 事件处理机制

12.2 组件事件

12.3 鼠标事件

12.4 键盘事件



12.1 事件处理机制

- **事件**：可以定义为程序发生了某些事情的信号。
- **事件源**：在其上发生事件的GUI组件被称为事件的源对象。例如，按钮是点击按钮事件的源对象。
- 一个事件是事件类的实例，**事件类的根类是**`java.util.EventObject`。
- 事件对象包含与事件有关的一切属性。可以使用`EventObject`类中的实例方法**`getSource()`**获得事件的源对象。



➤ 组件事件处理



触发事件

ActionEvent事件

调用并传递参数



12.2 组件事件

1. JButton按钮事件

单击按钮引发动作事件ActionEvent
对应的监听接口是ActionListener
对应的方法是actionPerformed

➤处理按钮事件的例子

点击按钮后程序打印出 “Button pressed!”

```
public void actionPerformed(ActionEvent e)
{
    if (e.getSource() == JButton)
        System.out.println ("Button pressed!");
}
```



2. JTextField文本框事件

ActionEvent, 当按回车时产生;

TextEvent, 当文本内容变化时产生

➤ 响应文本域的ActionEvent事件:

```
public void actionPerformed(ActionEvent  
e)
```

```
{
```

```
    if (e.getSource() == JTextField)
```

```
        ...
```

```
}
```



3.常用组件事件实例1




```
import java.awt.*; import java.awt.event.*;
import javax.swing.*;
public class test {    //主类
    public static void main(String args[ ]){
        new JFrameInOut( );    }
class JFrameInOut extends JFrame implements
                                ActionListener
{
    JLabel prompt;    //声明标签对象
    JTextField input,output;    //声明文本对象
    JButton btn;    //声明按钮对象
    JFrameInOut( ) {
        super("图形界面的Java Application程序");
        prompt=new JLabel("请输入您的姓名");
        input=new JTextField(6);
        output=new JTextField(20);
        btn=new JButton("关闭");    //产生按钮并初始化
```



```
setLayout(new FlowLayout()); //窗体布局
add(prompt);    add(input);
add(output);    add(btn);
input.addActionListener(this);
btn.addActionListener(this);
setSize(400, 400); //窗体大小
setVisible(true); //可见
```

```
}
public void actionPerformed(ActionEvent e) {
    if(e.getSource()==input) //动作控件是否为文本
        output.setText(input.getText()+" ,欢迎你!");
    else {
        dispose();
        System.exit(0); //关闭窗体
    }
}
```

4.常用组件事件实例2---弹出对话框

- 对话框**JDialog**实例必须要依赖于某个窗口。

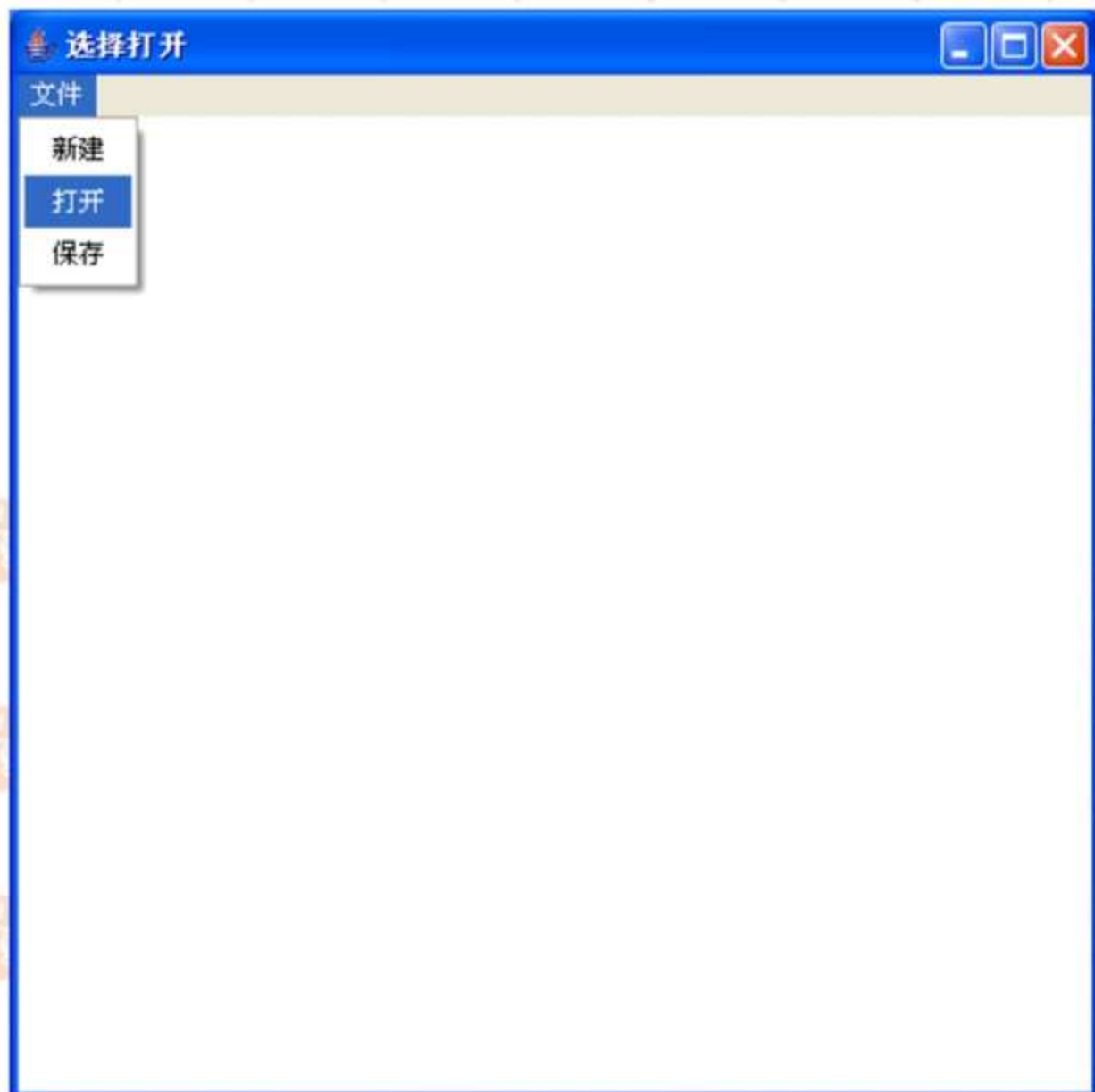
方法声明	方法说明
public JDialog ()	默认构造方法，构造一个无标题且没有指定所有者的无模式对话框
public JDialog (Frame owner)	构造方法，指定对话框的所有者为owner
public JDialog (Frame owner,String title,boolean modal)	构造方法，指定对话框的所有者、标题和模式
public Container getContentPane()	返回此对话框的 ContentPane 对象

➤ 消息对话框

- 消息对话框是有模式对话框，进行一个重要动作之前，最好能弹出一个消息对话框。
 - `public static void showMessageDialog(Component parent, Object message) throws HeadlessException`



5. 菜单事件



6.菜单事件举例



```
import java.awt.*;
import java.awt.event.*;import javax.swing.*;
class MenuExample extends JFrame implements ActionListener{
    JMenuBar menubar;    JMenu menu;
    JMenuItem itemNew,itemOpen,itemSave;
    MenuExample(String s){
        super(s);    setSize(500,500);
        menubar=new JMenuBar();
        menu=new JMenu("文件");
        itemNew=new JMenuItem("新建");
        itemOpen=new JMenuItem("打开");
        itemSave=new JMenuItem("保存");
        menu.add(itemNew);    menu.add(itemOpen);
        menu.add(itemSave);    menubar.add(menu);
        setJMenuBar(menubar); setVisible(true);
    }
}
```

```
this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    itemNew.addActionListener(this);  
    itemOpen.addActionListener(this);  
    itemSave.addActionListener(this);
```

```
public void actionPerformed(ActionEvent e) {  
    if(e.getSource()==itemNew)  
        this.setTitle("选择新建");  
    else if(e.getSource()==itemOpen)  
        this.setTitle("选择打开");  
    else if(e.getSource()==itemSave)  
        this.setTitle("选择保存");  }}
```

```
public class less {  
    public static void main(String args[]) {  
        MenuExample win=new MenuExample("菜单窗口")  
    }}
```



12.3 鼠标事件

1. MouseEvent 鼠标事件

- Java提供了处理鼠标事件的两个监听器接口：
`MouseListener` 和 `MouseMotionListener`。
- `MouseListener` 接口监听鼠标的按下、松开、进入、退出和点击等行为。
- `MouseMotionListener` 接口监听鼠标的移动和拖动等行为





- 当鼠标进入或离开组件时调用 `mouseEntered(MouseEvent e)` 和 `mouseExited(MouseEvent e)` 事件处理器。
- 当鼠标按下或松开时调用 `mousePressed(MouseEvent e)` 和 `mouseReleased(MouseEvent e)` 事件处理器。而当按下鼠标并松开后，调用 `mouseClicked(MouseEvent e)` 事件处理器。
- 当不按按钮移动鼠标时调用 `mouseMoved(MouseEvent e)` 事件处理器，当按下按钮移动鼠标时调用 `mouseDragged(MouseEvent e)` 事件处理器。



- 当鼠标进入到窗口时，背景色变为红色；
鼠标移出窗口后，背景色变为白色。



12.4 键盘事件

- 键盘接口:**KeyListener**
- Java在KeyEvent类中为许多键定义了常量，包括功能键在内，他们分别由
getKeyChar和**getKeyCode**方法获得；

➤ 方法：

getKeyChar() 获得键字符

getKeyCode() 获得键编码



```
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;
class KeyMessagePanel extends JPanel{
    int x=100,y=100;
    char s='A';
    public void paintComponent(Graphics g){
        super.paintComponent(g);
        //设置颜色
        g.setColor(Color.RED);
        //设置字体
        Font f=new Font("Courier",Font.ITALIC+Font.BOLD,24);
        g.setFont(f);
        g.drawString(String.valueOf(s),x,y);
    }
}
```



//myKeyListener监听器继承自键盘事件适配器

```
class myKeyListener extends KeyAdapter{
```

```
    KeyMessagePanel mp;
```

```
    public myKeyListener(KeyMessagePanel mp){
```

```
        this.mp=mp;
```

```
    }
```

```
    public void keyPressed(KeyEvent e){
```

```
        switch(e.getKeyCode()){
```

```
            case KeyEvent.VK_UP:mp.y-=10;break; //当箭头键向上
```

```
            case KeyEvent.VK_DOWN:mp.y+=10;break; //当箭头键向
```

下

```
            case KeyEvent.VK_LEFT:mp.x-=10;break; //当箭头键向左
```

```
            case KeyEvent.VK_RIGHT:mp.x+=10;break; //当箭头键向
```

右

```
        } mp.repaint();    }}
```





- **class KeyboardFrame extends JFrame {**
- **KeyMessagePanel mp=new KeyMessagePanel();**
- **KeyboardFrame(String str){**
- **super(str);**
- **setBounds(200,200,200,200);**
- **mp.setFocusable(true);**
- **this.getContentPane().add(mp);**
- **mp.addKeyListener(new myKeyListener(mp));**
- **setVisible(true);**
- **}**
- **}**
- **public class KeyboardFrameDemo{**
- **public static void main(String args[]){**
- **KeyboardFrame frame=new KeyboardFrame ("字母移动窗体");**
- **}**
- **}**



复习与预习

复习:

1. 常用组件事件
2. 鼠标事件
3. 键盘事件

预习: 异常

